

6 Lecture 6: Temporal-Difference Learning

Introduction

Lecture 5 中介绍了 Robbins-Monro (RM) Algorithm:

$$w_{k+1} = w_k - a_k \tilde{f}(w_k, \eta_k), \quad k = 1, 2, 3, \dots, \quad (6.1)$$

其中 w_k 为对解的第 k 次估计. Temporal-Difference Learning (TD learning) 算法思想与 RM 算法相似, 是强化学习中的一大类方法, 包含例如 basic TD, Sarsa, n -step Sarsa 和 Q-learning 算法在内的多种强化学习算法.

6.1 Basic TD: TD Learning of State Values

TD Learning of state values – Basic TD

与 Lecture 4 中的 Monte Carlo Learning 相似, 最简单的 TD learning 算法与 MC Basic 一样是应用于 Policy Iteration 中的 policy evaluation, 被称为 Basic TD. 回顾 Policy Iteration 算法如下:

1. **Policy Evaluation:** $v_{\pi_k} = r_{\pi_k} + \gamma \mathbf{P}_{\pi_k} v_{\pi_k}$, $v_{\pi_k}(s) = \sum_a \pi_k(a|s) \left(\sum_{r,s'} p(r,s'|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s') \right)$
 2. **Policy Improvement:** $\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_{r,s'} p(r,s'|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s') \right)}_{q_{\pi_k}(s,a)}, s \in \mathcal{S}$
- $$\Rightarrow \pi_{k+1}(a|s) = \begin{cases} 1, & a = a_k^*(s) \\ 0, & a \neq a_k^*(s) \end{cases}, \quad a_k^*(s) = \arg \max_a q_{\pi_k}(s,a).$$

MC Basic 是 model-free 算法, 其做法是从 (s,a) 开始根据 policy π_k 进行 N 次模拟 (trajectory), 并对它们的 return $g^{(i)}(s,a) = r_{t+1}^{(i)} + \gamma r_{t+2}^{(i)} + \gamma^2 r_{t+3}^{(i)} + \dots$ 取平均以逼近真实 action value $q_{\pi_k}(s,a)$:

$$q_{\pi_k}(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{N} \sum_{i=1}^N g^{(i)}(s,a). \quad (6.2)$$

与 MC Basic 一样, Basic TD 也是 model-free 算法, 依赖于数据, 即由给定的 policy π 生成的 state-reward 序列:

$$(s_0, r_1, s_1, \dots, s_t, r_{t+1}, s_{t+1}, \dots) \text{ or } \{(s_t, r_{t+1}, s_{t+1})\}_t. \quad (6.3)$$

Basic TD Algorithm. 但与 MC Basic 不同的是, Basic TD 不再等待所有采样结束后再进行估计, 而是借鉴 RM 算法增量更新 (incremental update) 的思想在数据收集过程中不断迭代. 根据 state value $v(s)$ 的定义, 有:

$$v_{\pi}(s_t) = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s_t] \Rightarrow f(v) = v(s_t) - (r_{t+1} + \gamma v(s_{t+1})) = 0. \quad (6.4)$$

其中 $f(v)$ 只使用了一条 trajectory, 因此是一个有噪观测. 借鉴 RM 算法 (6.1), 将上式 $f(v) = 0$ 的求解写成 RM 算法的增量更新形式就得到了 **Basic TD** 算法:

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t) [v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]], \quad (6.5a)$$

$$v_{t+1}(s) = v_t(s), \quad \forall s \neq s_t \text{ (未被访问的状态不更新, 有时省略此式)}. \quad (6.5b)$$

其中式 (6.5a) 可以更详细地拆解为:

$$\underbrace{v_{t+1}(s_t)}_{\text{new estimate}} = \underbrace{v_t(s_t)}_{\text{current estimate}} - \alpha_t(s_t) \left[\overbrace{v_t(s_t) - [r_{t+1} + \gamma v_t(s_{t+1})]}^{\text{TD error } \delta_t} \right]. \quad (6.6)$$

可以看出, 新的估计 $v_{t+1}(s_t)$ 是当前估计 $v_t(s_t)$ 和 TD error δ_t 的组合.

^a按照 [1] 中的写法, TD error $\delta_t \triangleq [r_{t+1} + \gamma v(s_{t+1})] - v(s_t) = \bar{v}_t - v(s_t)$.

Basic TD – TD Target and TD Error

TD Target. TD target 定义为:

$$\bar{v}_t \triangleq r_{t+1} + \gamma v(s_{t+1}). \quad (6.7)$$

称之为 **target** 是因为我们迭代的目标就是让 $v(s_t)$ 不断靠近 $\bar{v}_t$. 根据式(6.6), 可以得到

$$\begin{aligned} v_{t+1}(s_t) - \bar{v}_t &= v_t(s_t) - \bar{v}_t - \alpha_t(s_t) [v_t(s_t) - \bar{v}_t] = [1 - \alpha_t(s_t)] [v_t(s_t) - \bar{v}_t] \\ \implies |v_{t+1}(s_t) - \bar{v}_t| &= |1 - \alpha_t(s_t)| |v_t(s_t) - \bar{v}_t|. \end{aligned} \quad (6.8)$$

由于 discount rate $\gamma \in (0, 1)$, 因此有:

$$0 < \alpha_t(s_t) < 1 \Rightarrow 0 < 1 - \alpha_t(s_t) < 1 \Rightarrow |v_{t+1}(s_t) - \bar{v}_t| \leq |v_t(s_t) - \bar{v}_t|. \quad (6.9)$$

因此 $v_t(s_t) \rightarrow \bar{v}_t$.

TD Error. TD Error 反映了估计值 v_t 与真实状态值 v_π 之间的差异, 其定义为:

$$\delta_t \triangleq v(s_t) - [r_{t+1} + \gamma v(s_{t+1})] = v(s_t) - \bar{v}_t. \quad (6.10)$$

由于 $v_\pi(s_t) = \mathbb{E}[R_{t+1} + \gamma v_\pi(s_{t+1}) | S_t = s_t]$, 因此在期望意义下 $\delta_t = 0$, 即若 $v_t = v_\pi$, 则 $\delta_t = 0$:

$$\mathbb{E}[\delta_{\pi,t} | S_t = s_t] = v_\pi(s_t) - \mathbb{E}[R_{t+1} + \gamma v_\pi(s_{t+1}) | S_t = s_t] = 0. \quad (6.11)$$

Note 6.1. 可以看到 *TD error* 的定义和算法更新涉及到 t 与 $t + 1$ 两个时间步, 因此将此算法称为 “temporal difference”(时序差分), 反映了两个时间步间的差异.

Basic TD – Idea

Bellman expectation equation. 回顾 element-wise form 的 Bellman Equation 如下:

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a|s) \left[\sum_{r, s'} p(r, s' | s, a) r + \gamma \sum_{s'} p(s' | s, a) v_\pi(s') \right], \quad \forall s \in \mathcal{S}. \quad (6.12)$$

Bellman expectation equation 与其形式等价, 只是使用了更简洁的数学期望的形式. 根据 **state value** 的定义:

$$v_\pi(s) = \mathbb{E}[R + \gamma G | S = s], \quad s \in \mathcal{S}, \quad (6.13)$$

其中 G 为 **discounted return**. 由于 $\mathbb{E}[G | S = s] = \sum_a \pi(a|s) \sum_{s'} p(s' | s, a) v_\pi(s') = \mathbb{E}[v_\pi(S') | S = s]$, 因此:

$$v_\pi(s) = \mathbb{E}[R + \gamma v_\pi(S') | S = s], \quad s \in \mathcal{S}. \quad (6.14)$$

RM algorithm for solving the Bellman expectation equation (6.14). 首先定义求解的目标:

$$g(v(s)) = v(s) - v_\pi(s) = v(s) - \mathbb{E}[R + \gamma v_\pi(S') | S = s] \Rightarrow g(v(s)) = 0. \quad (6.15)$$

进而根据 R 和 S' 的采样 r, s' 得到有噪观测:

$$\tilde{g}(v(s)) = \underbrace{(v(s) - \mathbb{E}[R + \gamma v_\pi(S') | s])}_{g(v(s))} + \underbrace{(\mathbb{E}[R + \gamma v_\pi(S') | s] - [r + \gamma v_\pi(s')])}_{\eta}. \quad (6.16)$$

因此求解问题 $g(v(s)) = 0$ 的 RM 算法为:

$$v_{t+1}(s) = v_t(s) - \alpha_t \tilde{g}(v_t(s)) = v_t(s) - \alpha_t (v_t(s) - [r_t + \gamma v_\pi(s'_t)]), \quad (6.17)$$

其中 $v_t(s)$ 为 $v_\pi(s)$ 在第 t 步的估计, r_t, s'_t 分别为 R, S' 在第 t 步的采样.

Note 6.2. ① 式 (6.17) 仅对一个 *state* s 进行估计, 在 *TD learning* 中使用序列 $\{s_t, r_{t+1}, s_{t+1}\}$ 作为数据, 因此在当前时间步 t 没有涉及到的 *state* 就保持不变 (如式 6.5b). ② 算法 (6.17) 中使用了我们估计的目标 v_π 进行迭代, 在 *TD learning* 中由于 v_π 未知, 因此需要使用 $v_t(s')$ 替换 $v_\pi(s'_t)$. 此时依然有收敛性, 详见 **Theorem ??**.

Note 6.3. ① *TD learning* 使用 RM 算法求解策略 π 的 *Bellman expectation equation*, 但将估计目标 v_π 替换为当前状态下的估计 $v_t(s'_t)$. ② *TD learning* 的基本思想是根据新获得的信息纠正当前对状态值的估计, 从而使 $v_t \rightarrow v_\pi$.

TD Learning of state values: Convergence analysis

Theorem 6.1 (Convergence of Basic TD learning). 给定 policy π , 使用 Basic TD algorithm (6.5a)(6.5b)时, 若满足如下条件则 $\forall s \in \mathcal{S}, v_t(s) \xrightarrow[t \rightarrow \infty]{w.p.1} v_\pi(s)$:

- ① $\sum_t \alpha_t(s) = \infty$ (每一个状态 s 都会被访问足够多/无穷次, 因为未被访问时 $\alpha_t(s) = 0$)
- ② $\sum_t \alpha_t^2(s) < \infty, \forall s \in \mathcal{S}$ (意味着 $\alpha_t \rightarrow 0$, 实际中设为随着 t 最后变得很小的数)

Note 6.4. 事实上当 α 为常数时, 仍然可以证明算法在期望意义上收敛. **Theorem 6.1** 的证明详见附录??, 其需要使用 Lecture 5 中 Dvoretzky's theorem 的推广版本.

6.2 Sarsa: TD Learning of Action Values

TD Learning of action values – Sarsa

Sarsa 算法的目标是估计给定 policy π 的 action value $q_\pi(s, a)$, 因此与估计 state value 的 Basic TD 需要的数据 $\{(s_t, r_t, s_{t+1})\}_t$ 不同, 其需要的数据是 $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$.

根据 action value 的定义, 有:

$$q_\pi(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_\pi(s') = \mathbb{E}[R + \gamma q_\pi(S', A')|s, a], \quad \forall s, a. \quad (6.18)$$

因此与 Basic TD 一样, 定义求解目标为:

$$g(q(s)) = q(s) - q_\pi(s) = v(s) - \mathbb{E}[R + \gamma q_\pi(S', A')|s, a] \Rightarrow g(q(s)) = 0. \quad (6.19)$$

再应用 RM 算法并将未知的 $q_\pi(s', a')$ 替换成下一时间步的观测 $q_t(s_{t+1}, a_{t+1})$ 可得 **Sarsa** 算法为:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})]], \quad (6.20a)$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t), t = 0, 1, 2, \dots \quad (6.20b)$$

其中 $q_t(s_t, a_t)$ 是对 $q_\pi(s_t, a_t)$ 的估计, α_t 为 learning rate.

TD Target. Sarsa 算法的 TD target 为:

$$\bar{q}_t \triangleq r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}). \quad (6.21)$$

TD Error. Sarsa 算法的 TD error 为估计值 q_t 与真实状态值 q_π 之间的差异, 其定义为:

$$\delta_t \triangleq v(s_t) - [r_{t+1} + \gamma v(s_{t+1})] = v(s_t) - \bar{v}_t. \quad (6.22)$$

Note 6.5. ① Sarsa 使用的数据为 $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$, 首字母的缩写即构成了 *s-a-r-s-a* 这个命名. ② 类似于 TD learning, Sarsa 在求解 action value 版本的 Bellman equation (6.18), 证明其为 BE 可详见附录??.

Theorem 6.2 (Convergence of Sarsa). 给定 policy π , 若满足如下条件则 $\forall s \in \mathcal{S}, a \in \mathcal{A}, q_t(s, a) \xrightarrow[t \rightarrow \infty]{w.p.1} q_\pi(s, a)$:

- ① $\sum_t \alpha_t(s, a) = \infty$ (每对 (s, a) 都会被访问很多/无穷次, 因为未被访问时 $\alpha_t(s, a) = 0$)
- ② $\sum_t \alpha_t^2(s, a) < \infty, \forall s \in \mathcal{S}, a \in \mathcal{A}$ (意味着 $\alpha_t \rightarrow 0$, 实际中设置为随 t 逐渐变足够小)

Note 6.6. 收敛性的证明与 TD learning 中 Theorem ??类似, 因此暂略.

Sarsa – Algorithm

Sarsa 只能通过迭代得到给定 policy π 的 action value，其可用于 Policy Iteration 的 policy evaluation 部分计算 state value，后续仍需结合 policy improvement 才能得到 optimal policy。事实上 Sarsa 常常指代结合后完整的算法，即将 Policy Iteration 中的 policy evaluation 使用 Sarsa 计算 state value，后续继续结合 policy improvement。

Algorithm 7 Optimal policy learning by Sarsa

- 1: **Initialization:** $\alpha_t(s, a) = \alpha > 0$ for all (s, a) and all t . $\epsilon \in (0, 1)$. Initial $q_0(s, a)$ for all (s, a) . Initial ϵ -greedy policy π_0 derived from q_0 .
- 2: **Goal:** Learn an optimal policy that can lead the agent to the target state from an initial state s_0 .
- 3: **for each episode do**
- 4: **if** the current s_t is not the target state **then**
- 5: Collect the experience $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$: In particular, take action a_t following $\pi_t(s_t)$, generate r_{t+1}, s_{t+1} , and then take action a_{t+1} following $\pi_t(s_{t+1})$.
- 6: Update q -value(policy evaluation):

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}))]$$

- 7: Update policy(policy improvement):

$$\pi_{t+1}(a | s_t) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A|} & \text{if } a = \arg \max_a q_{t+1}(s_t, a) \\ \frac{\epsilon}{|A|} & \text{otherwise} \end{cases}$$

- 8: **end if**
- 9: **end for**

Note 6.7. 注意在做 policy evaluation 时只做了一次就进行了 policy improvement 并继续迭代，并没有准确进行估计，因此是基于 generalized policy iteration 的想法。

6.3 Expected Sarsa: TD Learning of Action Values

TD Learning of action values: Expected Sarsa

在 Sarsa 中 TD target (6.21) 包含下一次观测的 action value $q_t(s_{t+1}, a_{t+1})$ ，然而一次观测的方差过大，为了降低方差可以进行多次采样，将 $r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$ 变为 $r_{t+1} + \gamma \mathbb{E}[q_t(s_{t+1}, A)]$ ，就得到了 **Expected Sarsa** 算法：

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - (r_{t+1} + \gamma \mathbb{E}[q_t(s_{t+1}, A)])], \quad (6.23a)$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t). \quad (6.23b)$$

其中 $\mathbb{E}[q_t(s_{t+1}, A)] = \sum_a \pi_t(a | s_{t+1}) q_t(s_{t+1}, a) \triangleq v_t(s_{t+1})$ 。

Note 6.8. Expected Sarsa 需要更大的计算量去估计 $\mathbb{E}[q_t(s_{t+1}, A)]$ ，因此相较于 Sarsa 计算量增大，但是由于使用的随机变量的个数变少(由 S, A 变为 S)，因此估计方差变小。

Note 6.9. 与 Basic TD 和 Sarsa 一样，Expected Sarsa 可以解释为使用 RM 算法求解如下方程：

$$q_\pi(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \mathbb{E}_{A_{t+1} \sim \pi(s_{t+1})} [q_\pi(s_{t+1}, A_{t+1})] | S_t = s, A_t = a \right], \quad \forall s, a. \quad (6.24)$$

由于

$$\mathbb{E} [q_\pi(s_{t+1}, A_{t+1}) | S_{t+1}] = \sum_{A'} q_\pi(s_{t+1}, A') \pi(A' | S_{t+1}) = v_\pi(s_{t+1}) \quad (6.25)$$

因此式(6.24)是如下 Bellman equation 的另一种展开式：

$$q_\pi(s, a) = \mathbb{E} [R_{t+1} + \gamma v_\pi(s_{t+1}) | S_t = s, A_t = a] \quad (6.26)$$

6.4 n -Step Sarsa: TD Learning of Action Values

TD Learning of action values: n -step Sarsa

Sarsa (6.20a)(6.20b) 使用的样本只需要相邻时间步的 **state-action-reward pair** $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$, 而 Monte Carlo 则使用完整的 **trajectory** $(s_t, a_t, r_{t+1}, \dots)$ 进行估计. n -step Sarsa 是 Sarsa 的推广, 介于二者之间.

根据定义, **action value** $q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$, 其中 G_t 为 **discounted return**, 可以被写为多种形式:

$$\begin{aligned} \text{Sarsa} \leftarrow & \quad G_t^{(1)} = R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}), \\ & \quad G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, A_{t+2}), \\ & \quad \dots \\ \text{\textit{n}-step Sarsa} \leftarrow & \quad G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^n q_\pi(S_{t+n}, A_{t+n}), \\ \text{MC} \leftarrow & \quad G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \end{aligned} \quad (6.27)$$

Note 6.10. 所有含有上标的 $G_t^{(1)}, \dots, G_t^{(\infty)}$ 都是等价的, 不同的仅是如何分解. $G_t^{(\infty)}$ 中 ∞ 指达到 *episode* 终点.

得到 G_t 不同的分解后, 将其代入 **action value** 定义式中, 可以得到不同算法的求解目标:

$$\begin{aligned} \text{Sarsa to solve:} \quad & q_\pi(s, a) = \mathbb{E}[G_t^{(1)} | s, a] = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | s, a] \\ \text{\textit{n}-step Sarsa:} \quad & q_\pi(s, a) = \mathbb{E}[G_t^{(n)} | s, a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \dots + \gamma^n q_\pi(S_{t+n}, A_{t+n}) | s, a] \\ \text{MC learning:} \quad & q_\pi(s, a) = \mathbb{E}[G_t^{(\infty)} | s, a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | s, a] \end{aligned} \quad (6.28)$$

相应的随机近似算法如下:

$$\begin{aligned} \text{Sarsa:} \quad & q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})]] \\ \text{\textit{n}-step Sarsa:} \quad & q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) [q_t(s_t, a_t) - [r_{t+1} + \dots + \gamma^n q_t(s_{t+n}, a_{t+n})]] \\ \text{MC learning:} \quad & q_{t+1}(s_t, a_t) = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^n r_{t+n} + \dots \end{aligned} \quad (6.29)$$

从统一的角度来看, n -step Sarsa 的 **target** 是:

$$G_{t:t+n} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n Q_t(S_{t+n}, A_{t+n}) = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q_t(S_{t+n}, A_{t+n}). \quad (6.30)$$

更新为:

$$Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) + \alpha_t [G_{t:t+n} - Q_t(S_t, A_t)] = \underbrace{R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n}}_{\text{真实采样得到的奖励}} + \underbrace{\gamma^n Q_t(S_{t+n}, A_{t+n})}_{\text{bootstrap 估计}}. \quad (6.31)$$

因此 n 决定了采样多少步真实奖励, 然后在什么位置停止采样并使用当前 Q 值进行 **bootstrap**. 当 $n = 1$ 时, n -step Sarsa 退化为 **one-step Sarsa**; 当 n 延伸至 *episode* 终点时, 其 **target** 退化为完整 Monte Carlo return.

Bias Analysis. 真实的 **action value** 定义为:

$$q_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n q_\pi(S_{t+n}, A_{t+n}) \mid S_t = s, A_t = a \right]. \quad (6.32)$$

因此 TD **target** 与 真实 **action value** 的差异为:

$$\begin{aligned} \mathbb{E}_\pi [G_{t:t+n} \mid S_t = s, A_t = a] - q_\pi(s, a) &= \gamma^n \mathbb{E}_\pi [Q_t(S_{t+n}, A_{t+n}) - q_\pi(S_{t+n}, A_{t+n}) \mid S_t = s, A_t = a], \\ \Rightarrow |\mathbb{E}[G_{t:t+n} \mid s, a] - q_\pi(s, a)| &\leq \gamma^n \|Q_t - q_\pi\|_\infty. \end{aligned} \quad (6.33)$$

因此当 n 较小时, 较早使用不准确的 Q_t **bootstrap** 通常会造成偏差较大; 当 n 较大时, **bootstrap** 误差被 γ^n 衰减, 偏差通常较小.

Variance Analysis. 当 n 较小时, **target** 只包含少量随机奖励, 虽然 **bootstrap** 值可能不准确, 但它通常比一长串随机奖励更加稳定; 当 n 较大时, **target** 包含更多随机变量, 因此方差通常上升.

***n*-step Sarsa: Comparison**

下面比较这三种算法的不同之处：① **TD target:** Sarsa 与 *n*-step Sarsa 的 TD target 是不同的：

$$\begin{aligned} \text{Sarsa: } \bar{v}_t^{(1)} &\triangleq r_{t+1} + \gamma v(s_{t+1}) \\ n\text{-step Sarsa: } \bar{v}_t^{(2)} &\triangleq r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n}) \end{aligned} \quad (6.34)$$

② **Data needed:** 三种算法都是 model free 的算法，都需要数据，但所需数据不同：

$$\begin{aligned} \text{Sarsa: } &(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}) \\ n\text{-step Sarsa: } &(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \cdots, r_{t+n}, s_{t+n}, a_{t+n}) \\ \text{TD learning: } &(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \cdots, r_{t+n}, s_{t+n}, a_{t+n}, \cdots) \end{aligned} \quad (6.35)$$

③ **Online vs. Offline:** Sarsa 为 online 的算法，可以使用当前时间节点的数据立即进行算法迭代，而 MC learning 为 offline 的算法，需要等待到足够多的数据才可以估算 action value. *n*-step Sarsa 则既不是 online 也不是 offline 的算法，或将其看作是特殊的 online 算法，因为其需要等到 $t + n$ 时刻才可以更新 (s_t, a_t) 的 action value：

$$q_{t+n}(s_t, a_t) = q_{t+n-1}(s_t, a_t) - \alpha_{t+n-1}(s_t, a_t) [q_{t+n-1}(s_t, a_t) - [r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_{t+n-1}(s_{t+n}, a_{t+n})]] \quad (6.36)$$

6.5 Q-Learning: TD Learning of Optimal Action Values

TD Learning of optimal action values: Q-learning

Sarsa 及其变体是在估计 action value，在 Policy Iteration 算法中还需要结合 policy evaluation 才能够进行 policy searching (如 Algorithm 7 所示). 下面介绍非常经典且常用^a的算法 Q-learning，其直接估计 optimal action value，无需交替进行 policy evaluation 和 policy improvement. 为实现这一点，只需将 TD target 修改为 optimal：

$$\begin{aligned} \text{Q-learning: } &r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a) \\ \text{Sarsa: } &r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1}) \end{aligned} \quad (6.37)$$

因此 Q-learning 算法如下：

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - [r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a)] \right] \quad (6.38a)$$

$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t) \quad (6.38b)$$

Note 6.11. 与 Sarsa 在求解一个 action value 的 Bellman equation (6.18) 不同，Q-learning 是在求解一个带有期望的带有 action value 形式的 Bellman optimality equation (BOE) (6.39). 关于下式 (6.39) 是 BOE 的证明详见附录??

$$q(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_a q(S_{t+1}, a) \mid S_t = s, A_t = a \right], \quad \forall s, a. \quad (6.39)$$

^a现在常用的算法是 deep Q-learning，是将 Q-learning 与深度学习结合。

Behavior Policy and Target Policy

Definition 6.1 (Two policies in TD learning). TD learning 中存在两种 policy，分别为 **behavior policy** 和 **target policy**. 其中 behavior policy 用于生成 experience samples，是智能体与环境的交互策略，可以包含探索成分；target policy 定义我们希望估计的 value $q_\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a]$.

Note 6.12. ① Behavior policy 通常记作 b ，agent 真正与环境交互时 $A_t \sim b(\cdot \mid S_t)$ ，然后环境产生 $R_{t+1}, S_{t+1} \sim p(\cdot, \cdot \mid S_t, A_t)$. 因此 b 决定：(1) 实际执行哪个动作 (2) 哪些状态动作对被访问 (3) 数据集或 trajectory 的概率分布 (4) 探索程度是否充分. 例如 ϵ -greedy behavior policy $b(a \mid s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|}, & a \in \arg \max_{a'} Q(s, a'), \\ \frac{\epsilon}{|\mathcal{A}|}, & \text{otherwise} \end{cases}$ 偶尔执行

非 greedy 动作以保证探索. ② Target policy 通常记作 π ，用于定义我们希望估计的 value，其不一定真的在环境中执行，可以只存在于更新公式中。

Off Policy and On Policy

基于 target policy 和 behavior policy 是否一致可以将算法分为 on-policy 和 off-policy 两类.

Definition 6.2 (on-policy and off-policy). 当一种学习方法的 *behavior policy* 与 *target policy* 一致时, 称之为 *on-policy*, 即智能体在学习过程中是通过执行自己的 *target policy* 来收集数据和更新策略; 当 *behavior policy* 与 *target policy* 不一致时, 称之为 *off-policy*, 智能体可以通过 *behavior policy* 收集数据, 而通过 *target policy* 来进行学习.

Note 6.13. ① 使用 *off-policy* 的好处是其可以使用别的 (例如探索性更强的) *policy* 生成的 *experience samples* 去搜索 *optimal policy*, 可以加速学习进程. ② 区分算法是 *off-policy* 还是 *on-policy* 主要通过 (1) 明确求解目标 (例如是何种 BE/BOE) (2) 检查算法需要什么数据.

Sarsa is on-policy. Sarsa 在每个 iteration 中, 第 ① 步需要通过求解给定 policy π 的 Bellman Equation 进行 policy evaluation. 这一步 Sarsa 使用由 policy π 产生的数据估计 action value, 因此 π 为 behavior policy. 第 ② 步根据 π 的估计值进行 policy improvement π 也是 target policy.

Sarsa 需要的 data 为 $\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$, 其产生的方式为:

$$s_t \xrightarrow{\pi_b} a_t \xrightarrow{\text{model}} r_{t+1}, s_{t+1} \xrightarrow{\pi_b} a_{t+1}. \quad (6.40)$$

可以看到数据是通过 behavior policy π_b 产生的, 因此使用这些数据估计的 target 是:

$$Y_t^{\text{Sarsa}} = R_{t+1} + \gamma Q_t(S_{t+1}, A_{t+1}), \quad A_{t+1} \sim \pi_b(\cdot | S_{t+1}). \quad (6.41)$$

因此 Sarsa 实际求解的是 behavior policy π_b 对应的 action value 版本 Bellman Equation:

$$Q(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \sum_{a'} \pi_b(a' | S_{t+1}) Q(S_{t+1}, a') \mid s, a \right]. \quad (6.42)$$

而 Sarsa 的目的是估计 target policy π 的 action value, 其满足:

$$q_\pi(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \sum_{a'} \pi(a' | S_{t+1}) q_\pi(S_{t+1}, a') \mid s, a \right]. \quad (6.43)$$

要通过采样实现这个期望, 应当令 $A_{t+1} \sim \pi(\cdot | S_{t+1})$, 而实际中使用的是 π_b , 因此需要 $\pi_b = \pi$.

MC learning is on-policy. MC learning 的目标是求 action value 的期望:

$$q_\pi(s, a) = \mathbb{E} [R_{t+1} + \gamma R_{t+2} + \dots | S_t = s, A_t = a], \quad \forall s, a. \quad (6.44)$$

在实际算法中, 根据 behavior policy π_b 使用多个采样来近似估算 $q(s, a)$:

$$q(s, a) \approx r_{t+1} + \gamma r_{t+2} + \dots \quad (6.45)$$

为使得使用的数据所估算的 $q(s, a)$ 与目标 $q_\pi(s, a)$ 对齐, 需要 $\pi_b = \pi$, 因此 MC learning 为 on-policy.

Q-learning is off-policy. Q-learning 实际上在求解一个 BOE (6.39):

$$q(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q(S_{t+1}, a') \mid S_t = s, A_t = a \right], \quad \forall s, a. \quad (6.46)$$

其 target 为:

$$Y_t^Q = R_{t+1} + \gamma \max_{a'} Q_t(S_{t+1}, a'). \quad (6.47)$$

其中 $\max_{a'} Q_t(S_{t+1}, a')$ 假设未来按照 greedy policy 行动, 因此 π 为 greedy policy:

$$\pi(a | s) = \mathbf{1} \left\{ a \in \arg \max_{a'} Q_t(s, a') \right\}. \quad (6.48)$$

Q-learning 需要的数据为 $(s_t, a_t, r_{t+1}, s_{t+1})$, 其中若 (s_t, a_t) 给定, 那么 r_{t+1} 和 s_{t+1} 依赖于 model 而不依赖于 policy. 因此产生数据时, 通常使用探索性的 behavior policy: $b_t = \epsilon$ -greedy. 因此 $b_t = \epsilon$ -greedy $\neq \pi_t =$ greedy.

Note 6.14. 有一些方法可以将 on-policy 算法转换为 off-policy 算法, 例如重要性采样 (importance sampling).

Online and Offline

容易与 on-policy/off-policy 混淆的两个概念是 online/offline. 在线学习 (online) 是指 agent 在与环境交互过程中更新 value 和 policy. 离线学习 (offline) 是指 agent 使用预先收集的数据更新 value 和 policy 而无需与环境交互. 因此若一个算法是 on-policy 的, 那么它只能以 online 的模式进行, 不能使用别的 policy 预先产生的数据; 若算法是 off-policy 的, 那么它 online / offline 的模式都可以使用.

Q-learning – Algorithm

由于 Q-learning 是 off-policy 的, 其 behavior policy 与 target policy 可以不同也可以相同, 分别能够得到如下 online version 与 offline version 两种版本的算法:

Algorithm 8 Policy Searching by Q-learning (on-policy version)

- 1: **for** each episode **do**
- 2: **if** the current s_t is not the target state **then**
- 3: Collect the experience $(s_t, a_t, r_{t+1}, s_{t+1})$: In particular, take action a_t following $\pi_t(s_t)$, generate r_{t+1}, s_{t+1} .
- 4: **Update q-value:**

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - \left(r_{t+1} + \gamma \max_a q_t(s_{t+1}, a) \right) \right]$$

- 5: **Update policy:**

$$\pi_{t+1}(a | s_t) = \begin{cases} 1 - \frac{\epsilon}{|A|-1} & \text{if } a = \arg \max_a q_{t+1}(s_t, a), \\ \frac{\epsilon}{|A|} & \text{otherwise.} \end{cases}$$

- 6: **end if**
- 7: **end for**

Algorithm 9 Optimal Policy Search by Q-learning (off-policy version)

- 1: **for** each episode $\{s_0, a_0, r_1, s_1, a_1, r_2, \dots\}$ generated by π_b **do** $\triangleright \pi_b$ is behavior policy
- 2: **for** each step $t = 0, 1, 2, \dots$ of the episode **do**
- 3: **Update q-value:**

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - \left(r_{t+1} + \gamma \max_a q_t(s_{t+1}, a) \right) \right]$$

- 4: **Update target policy:**

$$\pi_{T,t+1}(a | s_t) = \begin{cases} 1 & \text{if } a = \arg \max_a q_{t+1}(s_t, a), \\ 0 & \text{otherwise.} \end{cases}$$

$\triangleright \pi_T$ is target policy

- 5: **end for**
- 6: **end for**

Note 6.15. 在 off-policy version 中直接使用 greedy 策略进行 target policy 的更新而无需具备探索性是因为此时数据由 behavior policy 生成, 只需要 behavior policy 保持探索性, 而 target policy 直接选择最好的即可; 相反在 on-policy version 中, 由于需要使用 target policy 生成数据, 因此需要保持一定的探索性, 故使用 ϵ -greedy 的更新模式.

6.6 Comparison and Conclusion

Comparison: TD Learning and MC Learning

Table 6.1: Comparison between TD/Sarsa learning and Monte Carlo learning

TD/Sarsa learning ¹	Monte Carlo learning
Online / incremental 每获得一步transition sample, 便可立即更新state/action-value estimate, 无需等待episode 结束.	Episode-wise / delayed 由于target 是完整的discounted return, 通常要等episode 采样结束后才能更新.
Episodic and continuing tasks 可自然用于episodic task, 也适用于没有terminal state 的continuing task.	Primarily episodic tasks 经典MC 方法依赖完整return, 因此主要用于具有terminal state 的episodic task.
Bootstrapping Update target 使用已有的value estimate, 例如Sarsa 的 $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$.	No bootstrapping Update target 直接采用实际采样得到的完整return G_t , 不依赖后继状态的value estimate.
Lower variance 一步TD target 涉及的随机样本较少; 例如Sarsa 只需采样 R_{t+1} 、 S_{t+1} 和 A_{t+1} , 通常具有较低方差.	Higher variance MC target $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$ 受后续整条trajectory 的随机性影响, 通常具有较高方差.
Possible finite-sample bias 由于target 依赖当前的近似估计, 有限样本下可能有偏; 在适当条件下可收敛到真实value.	Unbiased return target 对由policy π 生成的完整episode, G_t 是 $v_\pi(s_t)$ (或 $q_\pi(s_t, a_t)$) 的无偏样本.

¹ Sarsa 是典型的on-policy TD control 方法, 因此此处与TD learning 一并比较.

A Unified Point of View for TD Learning

本节中所介绍的所有算法实际上都可以表达为一个统一的形式:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - \bar{q}_t], \quad (6.49)$$

其中 $\bar{q}_t$ 为 TD target. 所有的 TD 算法的不同点在于 TD target 和数学上的求解目标:

Table 6.2: Update targets used by different reinforcement learning algorithms

Algorithm	Update target $\bar{q}_t$
Sarsa	$\bar{q}_t = R_{t+1} + \gamma q_t(S_{t+1}, A_{t+1})$
n -step Sarsa	$\bar{q}_t = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n q_t(S_{t+n}, A_{t+n})$
Expected Sarsa	$\bar{q}_t = R_{t+1} + \gamma \sum_a \pi_t(a S_{t+1}) q_t(S_{t+1}, a)$
Q-learning	$\bar{q}_t = R_{t+1} + \gamma \max_a q_t(S_{t+1}, a)$
Monte Carlo ¹	$\bar{q}_t = G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$

¹ 当 $\alpha_t(S_t, A_t) = 1$ 时, 有 $q_{t+1}(S_t, A_t) = \bar{q}_t$.

Table 6.3: Bellman equations solved by different reinforcement learning algorithms

Algorithm	Equation Solved
Sarsa	BE: $q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) S_t = s, A_t = a]$
n -step Sarsa	BE: $q_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n q_\pi(S_{t+n}, A_{t+n}) S_t = s, A_t = a \right]$
Expected Sarsa	BE: $q_\pi(s, a) = \mathbb{E}_\pi \left[R_{t+1} + \gamma \sum_{a'} \pi(a' S_{t+1}) q_\pi(S_{t+1}, a') S_t = s, A_t = a \right]$
Q-learning	BOE: $q_*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') S_t = s, A_t = a \right]$
Monte Carlo	BE: $q_\pi(s, a) = \mathbb{E}_\pi[G_t S_t = s, A_t = a]$